Backend Architecture Guide

Core Concepts

- A robust financial backend pairs Express middleware with PostgreSQL transactions to guarantee secure and consistent disbursement operations.
- Role-based access controls and server-side validation protect endpoints from malicious inputs and unauthorized authorization attempts.
- Offloading external API calls to background message queues like BullMQ prevents network latency from blocking the main web server thread.
- Cryptographic audit trails and pessimistic database locking ensure that transaction ledgers remain tamper-evident and safe from concurrent modifications.

Quick Check

- **Q:** Why must body-parsing middleware like `express.json()` run before route handlers?
  A: It ensures incoming JSON streams are fully parsed and attached to the request body before routes access them.
- **Q:** How do you prevent race conditions when multiple approvers modify a database record?
  A: Use pessimistic locking via `SELECT FOR UPDATE` to lock the rows until the active transaction completes.
- **Q:** Why must database transactions use a dedicated client from the connection pool?

A: Executing `BEGIN` and `COMMIT` on separate connections breaks isolation and causes silent data corruption.

- **Q:** How does a backend securely verify incoming payment success webhooks?
  A: By calculating an HMAC-SHA256 signature using a shared secret to authenticate the payload origin.
- **Q:** Why run database integration tests inside transactional blocks that always roll back?
  A: It prevents tests from mutating physical data, keeping the database state clean for subsequent assertions.
- **Q:** Which SQL clause prevents duplicate key errors during repeated local database seeding?
  A: Appending `ON CONFLICT DO NOTHING` to insert statements ensures idempotent script execution.

Key Terms

- **Role-Based Access Control:** A security mechanism implemented via middleware that restricts endpoint execution based on the authenticated user's assigned permissions.
- **Idempotent Seeding:** The practice of writing database population scripts that can safely run multiple times without causing duplicate record collisions.
- **Transaction Wrapper:** An isolation boundary using `BEGIN`, `COMMIT`, and `ROLLBACK` to ensure multiple database writes succeed or fail entirely together.
- **Tamper-Evident Ledger:** An audit trail where each new record incorporates the cryptographic hash of the previous record to instantly expose unauthorized alterations.
- **Asynchronous Worker:** An isolated background process that handles slow external tasks, like sending SMS alerts, without hanging the primary web API thread.

- **Server-Side Validation:** The essential security practice of explicitly parsing and rejecting malformed incoming request payloads on the backend before querying the database.